

实验报告

学生姓名： 汤承洲 学 号： 20233902058
专 业： 电子信息工程 年级、班级： 23 级 2 班
课程名称： 控制工程 实验项目： PID 控制器的设计
实验类型： ☒ 验证 ☐ 设计 ☐ 综合 实验时间： 2026 年 6 月 9 日
实验指导老师： 熊爱民 实验评分： _____

1. 实验目的

1. 研究 PID 控制器中比例系数 K_p 、积分时间常数 T_i 和微分时间常数 T_d 对闭环系统阶跃响应的影响。
2. 掌握 Ziegler-Nichols（齐格勒-尼科尔斯）工程整定方法，能够对典型被控对象进行 PID 参数整定。
3. 学会在 MATLAB 环境中使用 PID 控制器进行系统仿真，并通过性能指标（上升时间、超调量、调节时间等）评估控制效果。

2. 实验原理

2.1. 模拟 PID 控制器

典型的 PID 控制结构如图 1 所示。输入信号 $r(t)$ 与反馈信号 $y(t)$ 通过求和节点产生误差信号 $e(t)$ 。误差信号分为三路，分别进入比例、积分、微分三个环节，其输出相加得到控制信号 $u(t)$ ，作用于被控对象产生系统输出 $y(t)$ ，输出通过负反馈回路送回输入端。

图 1: PID 控制结构框图

PID 调节器的数学描述为：

$$u(t) = K_p \left[e(t) + \frac{1}{T_i} \int_0^t e(\tau) d\tau + T_d \frac{de(t)}{dt} \right]$$

其中， K_p 为比例系数， T_i 为积分时间常数， T_d 为微分时间常数。比例环节即时响应误差，积分环节消除稳态误差，微分环节预测误差变化趋势并增加系统阻尼。

2.2. 数字 PID 控制器

在计算机控制中，连续 PID 算法需经离散化处理。以一阶后向差分近似微分、矩形法数值积分近似积分：

$$\begin{cases} t \approx kT \\ \int_0^t e(\tau) d\tau \approx T \sum_{j=0}^k e(j) \\ \frac{de(t)}{dt} \approx \frac{e(k) - e(k-1)}{T} \end{cases}$$

离散 PID 表达式为：

$$u(k) = K_p \left[e(k) + \frac{T}{T_i} \sum_{j=0}^k e(j) + \frac{T_d}{T} (e(k) - e(k-1)) \right]$$

其中 T 为采样周期。

2.3. Ziegler-Nichols 整定法

Ziegler-Nichols 法是经典的 PID 参数工程整定方法。对于稳定的被控对象，先令 $T_i \rightarrow \infty$ 、 $T_d = 0$ ，仅使用比例控制，逐步增大 K_p 直至系统输出产生等幅振荡，记录此时的临界增益 K_c 和临界振荡周期 T_c 。然后按下表计算 PID 参数：

表 1: Ziegler-Nichols PID 参数整定公式

控制器类型	K_p	T_i	T_d
P	$0.5K_c$	∞	0
PI	$0.45K_c$	$0.85T_c$	0
PID	$0.6K_c$	$0.5T_c$	$0.125T_c$

3. 实验内容

本次实验以三阶对象 $G(s) = 1/(s+1)^3$ 为被控对象，分为两个任务：任务一研究 PID 参数对闭环系统阶跃响应的影响；任务二使用 Ziegler-Nichols 法进行

PID 参数整定并优化。

3.1. 任务一：PID 参数对闭环系统阶跃响应的影响

被控对象为三阶系统：

$$G(s) = \frac{1}{(s+1)^3}$$

3.1.1 (1) 纯比例控制：不同 K_p 值下的阶跃响应 ($T_i \rightarrow \infty, T_d \rightarrow 0$)

在 $T_i \rightarrow \infty, T_d \rightarrow 0$ 的条件下,系统退化为纯比例控制。取 $K_p = 0.5, 1, 2, 3, 4, 5$, 观察闭环系统的阶跃响应变化。

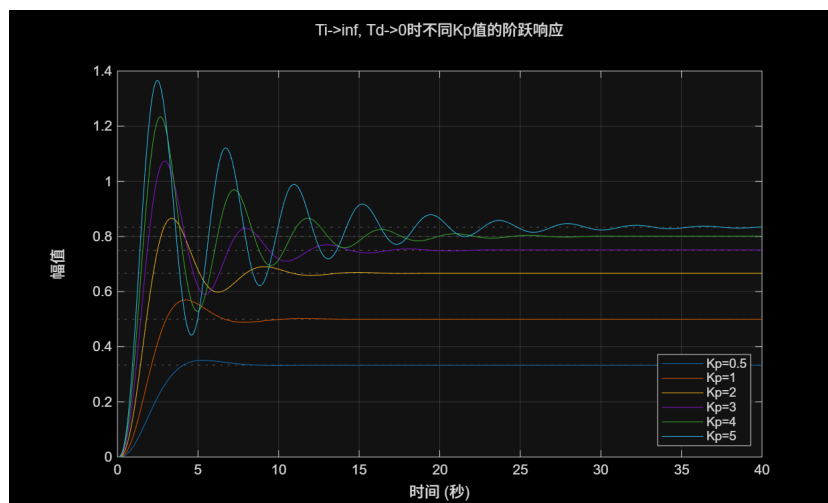


图 2: 不同 K_p 值下的阶跃响应 ($T_i \rightarrow \infty, T_d \rightarrow 0$)

结果分析：随着 K_p 增大，系统响应加快（上升时间缩短），但超调量逐渐增大。 $K_p = 0.5$ 时响应缓慢且无超调； $K_p = 3$ 时出现明显超调； $K_p = 5$ 时超调显著增大，系统趋于临界不稳定状态。纯比例控制存在稳态误差，且过大的 K_p 会恶化系统稳定性。

3.1.2 (2) PI 控制：不同 T_i 值下的阶跃响应 ($K_p = 1, T_d \rightarrow 0$)

固定 $K_p = 1, T_d \rightarrow 0$ ，取 $T_i = 0.5, 1, 2, 5, 10$ ，分析积分时间常数对系统响应的的影响。根据极点分布判断稳定性，将稳定与不稳定的曲线分别绘制。

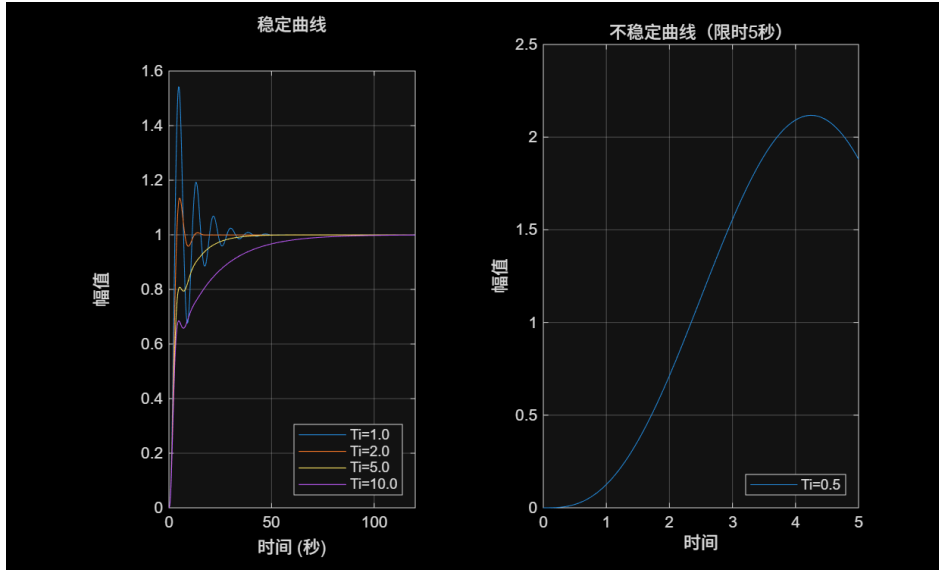


图 3: 不同 T_i 值下的阶跃响应 ($K_p = 1$, $T_d \rightarrow 0$): (左) 稳定曲线 (右) 不稳定曲线

结果分析: 当 $T_i = 0.5$ 时, 极点位于右半平面, 系统发散不稳定; 当 $T_i \geq 1$ 时系统稳定。在同一 K_p 下, T_i 越小积分作用越强, 稳态误差消除越快, 但系统稳定性变差; T_i 越大积分作用越弱, 响应曲线趋向于纯比例控制的效果。积分环节的引入能够消除稳态误差, 但会降低系统的稳定性裕度。

3.1.3 (3) PID 控制: 不同 T_d 值下的阶跃响应 ($K_p = T_i = 1$)

固定 $K_p = 1$ 、 $T_i = 1$, 取 $T_d = 0, 0.5, 1, 2, 3, 4$, 分析微分时间常数对系统响应的影响。

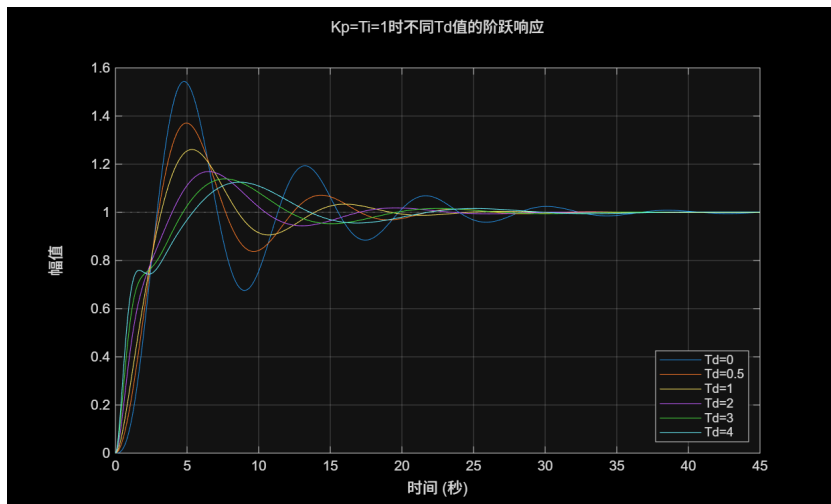


图 4: 不同 T_d 值下的阶跃响应 ($K_p = T_i = 1$)

结果分析：随着 T_d 增大，微分作用增强，系统阻尼增加，超调量显著减小，响应曲线趋于平缓。 $T_d = 0$ 时系统超调最大； $T_d = 0.5 \sim 1$ 时超调明显减小，响应速度和阻尼达到较好的平衡； $T_d = 3 \sim 4$ 时响应虽然平滑但上升时间明显延长。说明适当的微分作用可改善系统阻尼特性，但过大的微分会降低响应速度。

3.2. 任务二：PID 参数整定（Ziegler-Nichols 法）

3.2.1 系统的根轨迹分析

首先绘制被控对象 $G(s) = 1/(s+1)^3$ 的根轨迹图，分析系统的稳定性特征。

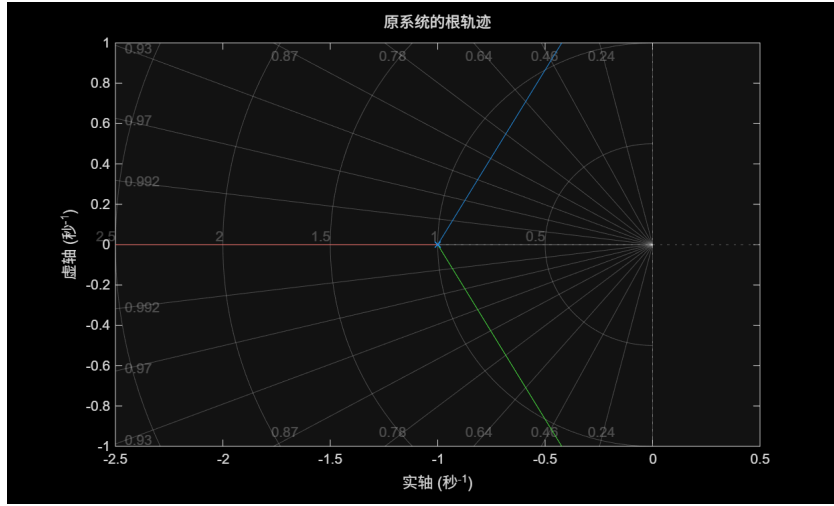


图 5: 原系统的根轨迹图

对于三阶对象 $G(s) = 1/(s+1)^3$ ，手动计算得到临界参数：

$$K_c = 8, \quad T_c = \frac{2\pi}{\sqrt{3}} \approx 3.6276 \text{ s}$$

3.2.2 Ziegler-Nichols PID 参数整定

根据 Z-N 整定公式计算 PID 参数：

$$K_p = 0.6K_c = 4.8000$$

$$T_i = 0.5T_c = 1.8138$$

$$T_d = 0.125T_c = 0.4534$$

使用整定后的 PID 控制器进行阶跃响应仿真：

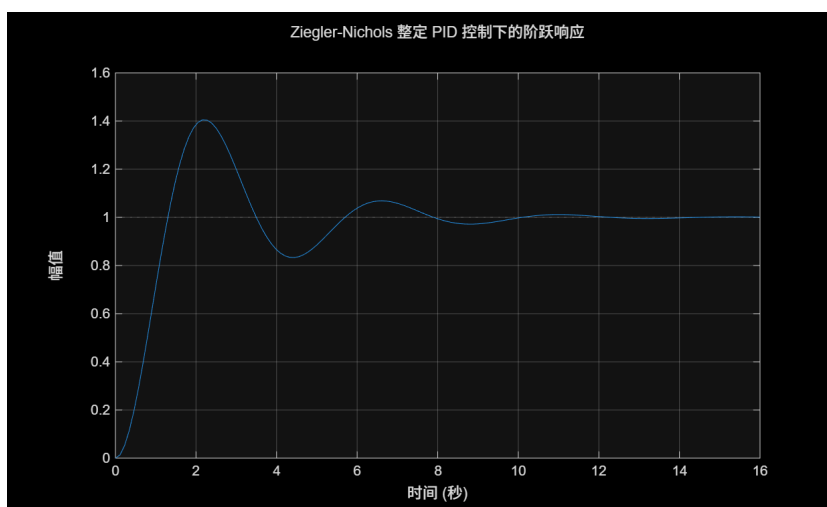


图 6: Ziegler-Nichols 整定 PID 控制下的阶跃响应

3.2.3 参数微调与性能对比

在 Z-N 参数基础上进行微调： K_p 降至 3.8400 ($0.8K_{p_zn}$)， T_i 降至 1.6324 ($0.9T_{i_zn}$)， T_d 增至 0.5441 ($1.2T_{d_zn}$)。

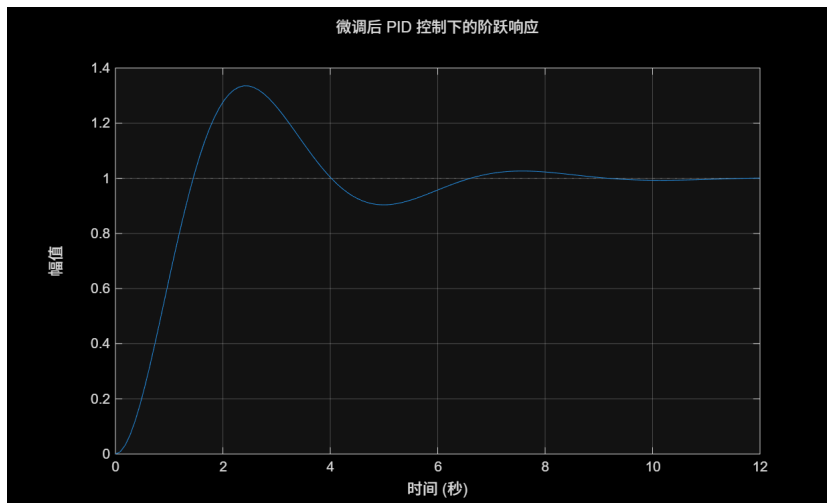


图 7: 微调后 PID 控制下的阶跃响应

性能指标对比如下：

表 2: Z-N PID 与微调后 PID 性能指标对比

指标	Z-N PID	微调后 PID	变化趋势
上升时间 (s)	0.8761	0.9831	略有增加
峰值时间 (s)	2.1641	2.3904	略有增加
超调量 (%)	40.49	33.52	显著减小
调节时间 (s)	9.3721	8.1661	明显缩短

结果分析：微调后的 PID 虽然上升时间略有增加（从 0.8761s 增至 0.9831s），但超调量从 40.49% 降至 33.52%，调节时间从 9.3721s 缩短至 8.1661s，综合性能优于 Z-N 整定参数。这表明适当降低 K_p 和 T_i 、增大 T_d 可以在抑制超调与缩短调节时间之间取得更好的平衡。

3.2.4 PID 各参数影响的进一步分析

固定其他参数为 Z-N 整定值，分别改变 K_p 、 T_i 、 T_d ，观察各参数对系统响应的的影响。

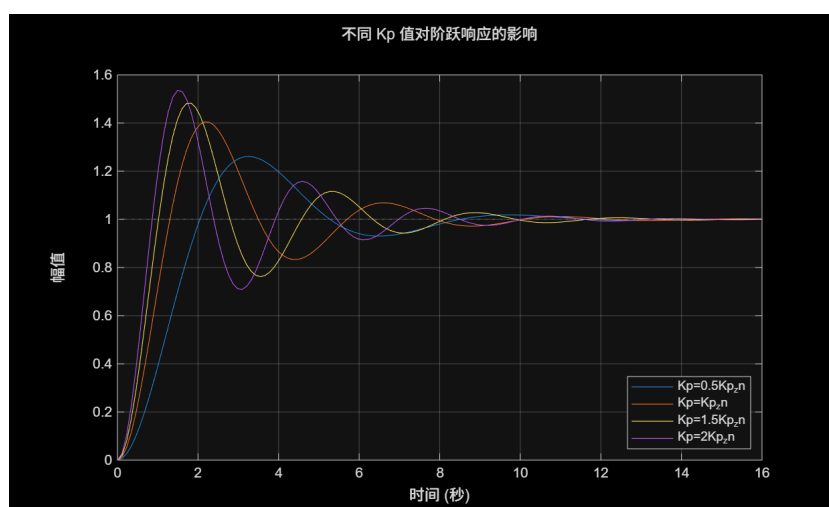


图 8: 不同 K_p 值对阶跃响应的影响

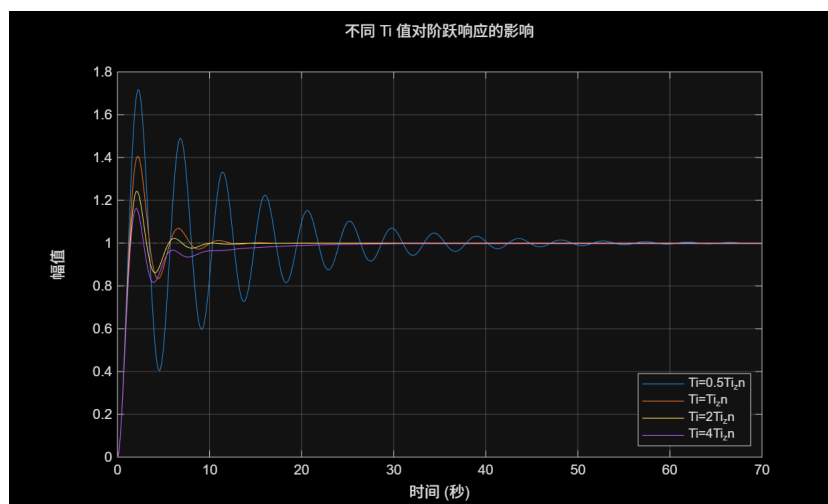


图 9: 不同 T_i 值对阶跃响应的影响

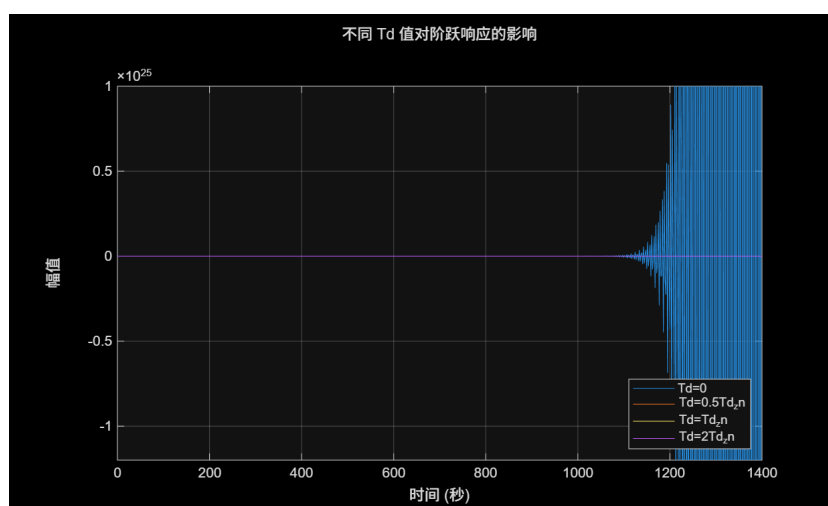


图 10: 不同 T_d 值对阶跃响应的影响

结果分析:

- K_p 的影响: 增大 K_p 使响应加快、上升时间缩短, 但超调量增大, 系统稳定性降低。 $K_p = 2K_{p_zn}$ 时系统出现明显的振荡。
- T_i 的影响: 减小 T_i 增强积分作用, 加快稳态误差消除, 但超调增大、稳定性下降; 增大 T_i 使响应趋近于纯比例控制, 稳态误差消除变慢。
- T_d 的影响: 增大 T_d 增强微分阻尼作用, 超调量减小, 系统稳定性改善, 但过大的 T_d 会显著降低响应速度。

3.2.5 最优参数选择

综合权衡各项性能指标, 选取最优参数 $K_p = 3.3600$ ($0.7K_{p_zn}$)、 $T_i = 1.4510$ ($0.8T_{i_zn}$)、 $T_d = 0.4534$ ($1.0T_{d_zn}$)。

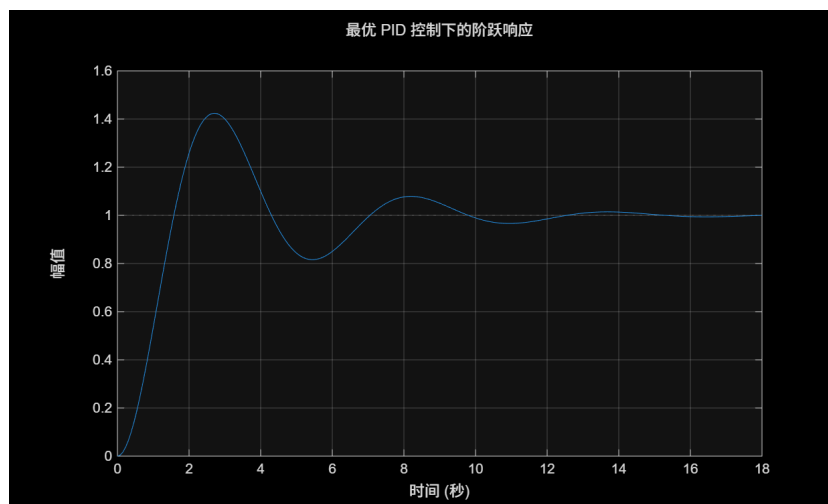


图 11: 最优 PID 控制下的阶跃响应

最优 PID 参数性能:

- 上升时间: 1.0593 s
- 超调量: 42.41%
- 调节时间: 11.8093 s

结果分析: 相比微调参数, 最优参数进一步降低了 K_p 和 T_i , 响应更加平稳但上升时间和调节时间变长。说明 PID 参数整定需要在快速性 (上升时间) 和稳定性 (超调量、调节时间) 之间进行折中, 不存在使所有指标同时最优的单一参数组合。

4. 实验总结

本次实验系统地研究了 PID 控制器参数对闭环系统动态性能的影响, 并掌握了 Ziegler-Nichols 工程整定方法, 主要收获如下:

4.1. PID 参数影响规律

1. 比例系数 K_p : 增大 K_p 可加快响应速度、缩短上升时间, 但会增大超调量、降低系统稳定性。过大的 K_p 可能导致系统不稳定。
2. 积分时间常数 T_i : 减小 T_i 增强积分作用, 有利于消除稳态误差, 但会增大超调、降低稳定性。当 T_i 过小时系统可能出现不稳定 (如本次实验中 $T_i = 0.5$ 时系统发散)。
3. 微分时间常数 T_d : 增大 T_d 增强微分阻尼, 可有效抑制超调, 改善系统稳定性, 但过大的 T_d 会降低响应速度。

4.2. Ziegler-Nichols 整定方法

1. Z-N 法通过临界增益 K_c 和临界振荡周期 T_c 快速获取 PID 参数初值: $K_p = 4.8000$ 、 $T_i = 1.8138$ 、 $T_d = 0.4534$, 所得系统上升时间快 (0.8761s) 但超调偏高 (40.49%)。
2. 在 Z-N 参数基础上进行微调 (降低 K_p 和 T_i 、增大 T_d), 可将超调量降至 33.52%, 调节时间缩短至 8.1661s, 综合性能优于原始 Z-N 参数。
3. 最优参数选择需在快速性与稳定性之间折中: 进一步降低 K_p 和 T_i 虽能提高稳定性, 但会牺牲响应速度。

4.3. 实验心得

通过本次实验, 加深了对 PID 控制器各参数物理意义的理解, 掌握了利用 MATLAB 进行控制系统仿真的方法, 以及通过时域性能指标 (上升时间、超调量、调节时间等) 定量评估控制效果的技能。Ziegler-Nichols 法提供了良好的参数初值, 实际工程中还需结合具体指标要求进行微调优化, 以达到最佳的控制性能。

5. 实验源码

5.1. task1.m: PID 参数对闭环系统阶跃响应的影响

```
1 % task1 研究闭环系统在不同控制情况下的阶跃响应
2 clc;
3 clear;
4 close all;
5 diary('results/logs/Task01_output.log');
6
7 % 三阶对象模型  $G(s) = 1 / (s + 1)^3$ 
8 s = tf('s');
9 G = 1 / (s + 1)^3;
10
11 %% 1.  $T_i \rightarrow \infty$ ,  $T_d \rightarrow 0$  时, 在不同  $K_p$  值下, 闭环系统的阶跃响应
12 figure;
13
14 Kp_list = [0.5, 1, 2, 3, 4, 5];
15 for i = 1:length(Kp_list)
16     Kp = Kp_list(i);
17     % 纯比例控制:  $T_i = \infty$ ,  $T_d = 0$ ,  $N = \infty$ 
18     C = pidstd(Kp, Inf, 0, Inf);
19     G_cl = feedback(C * G, 1);
20     step(G_cl);
21     hold on;
22 end
23
24 legend('Kp=0.5', 'Kp=1', 'Kp=2', 'Kp=3', 'Kp=4', 'Kp=5', 'Location', 'southeast');
25 title('Ti->inf, Td->0时不同Kp值的阶跃响应');
26 xlabel('时间');
27 ylabel('幅值');
28 grid on;
29 hold off;
30 saveas(gcf, 'results/figures/Task01_Figure_1.png');
31
32 %% 2.  $K_p = 1$ ,  $T_d \rightarrow 0$  时, 在不同  $T_i$  值下, 闭环系统的阶跃响应
33 Ti_list = [0.5, 1, 2, 5, 10];
34
35 % 判断稳定性, 分组合并
36 stable_Ti = [];
37 unstable_Ti = [];
38
39 for i = 1:length(Ti_list)
40     Ti = Ti_list(i);
41     C = pidstd(1, Ti, 0, Inf);
42     G_cl = feedback(C * G, 1);
43     poles = pole(G_cl);
```

```

44     if all(real(poles) < 0)
45         stable_Ti = [stable_Ti, Ti];
46     else
47         unstable_Ti = [unstable_Ti, Ti];
48     end
49 end
50
51 % 子图1: 稳定的曲线
52 figure;
53 subplot(1, 2, 1);
54 for i = 1:length(stable_Ti)
55     Ti = stable_Ti(i);
56     C = pidstd(1, Ti, 0, Inf);
57     G_cl = feedback(C * G, 1);
58     step(G_cl);
59     hold on;
60 end
61 legend(arrayfun(@(x) sprintf('Ti=%.1f', x), stable_Ti, 'UniformOutput'
62     , false), ...
63     'Location', 'southeast');
64 title('稳定曲线');
65 xlabel('时间');
66 ylabel('幅值');
67 grid on;
68 hold off;
69
70 % 子图2: 不稳定的曲线 (限时5秒)
71 subplot(1, 2, 2);
72 for i = 1:length(unstable_Ti)
73     Ti = unstable_Ti(i);
74     C = pidstd(1, Ti, 0, Inf);
75     G_cl = feedback(C * G, 1);
76     [y, t] = step(G_cl, 5);
77     plot(t, y);
78     hold on;
79 end
80 legend(arrayfun(@(x) sprintf('Ti=%.1f', x), unstable_Ti, '
81     UniformOutput', false), ...
82     'Location', 'southeast');
83 title('不稳定曲线 (限时5秒)');
84 xlabel('时间');
85 ylabel('幅值');
86 grid on;
87 hold off;
88 saveas(gcf, 'results/figures/Task01_Figure_2.png');
89
90 %% 3.  $K_p = T_i = 1$  时, 在不同  $T_d$  值下, 闭环系统的阶跃响应
91 figure;

```

```

91 Td_list = [0, 0.5, 1, 2, 3, 4];
92 for i = 1:length(Td_list)
93     Td = Td_list(i);
94     C = pidstd(1, 1, Td, Inf);
95     G_cl = feedback(C * G, 1);
96     step(G_cl);
97     hold on;
98 end
99
100 legend('Td=0', 'Td=0.5', 'Td=1', 'Td=2', 'Td=3', 'Td=4', 'Location', 'southeast');
101 title('Kp=Ti=1时不同Td值的阶跃响应');
102 xlabel('时间');
103 ylabel('幅值');
104 grid on;
105 hold off;
106 saveas(gcf, 'results/figures/Task01_Figure_3.png');
107
108 diary off;

```

5.2. task2.m: PID 参数整定 (Ziegler-Nichols 法)

```

1 % task2 选择合适的参数进行模拟 PID 控制
2 clc;
3 clear;
4 close all;
5 diary('results/logs/Task02_output.log');
6
7 % 三阶对象模型  $G(s) = 1 / (s + 1)^3$ 
8 s = tf('s');
9 G = 1 / (s + 1)^3;
10
11 % 手动计算临界增益 (对于  $G(s)=1/(s+1)^3$ )
12 Kc = 8;
13 Tc = 2 * pi / sqrt(3); % 3.6276
14
15 fprintf('临界增益 Kc = %.4f\n', Kc);
16 fprintf('临界振荡周期 Tc = %.4f 秒\n', Tc);
17
18 % Ziegler-Nichols PID 参数 (标准型)
19 Kp_zn = 0.6 * Kc;
20 Ti_zn = 0.5 * Tc;
21 Td_zn = 0.125 * Tc;
22
23 fprintf('\nZiegler-Nichols 整定参数:\n');
24 fprintf('Kp = %.4f\n', Kp_zn);
25 fprintf('Ti = %.4f\n', Ti_zn);
26 fprintf('Td = %.4f\n', Td_zn);

```

```

27
28 %% 1. 根轨迹图
29 figure;
30 rlocus(G);
31 title('原系统的根轨迹');
32 grid on;
33 saveas(gcf, 'results/figures/Task02_Figure_1.png');
34
35 %% 2. 使用整定后的 PID 控制器进行仿真
36 figure;
37
38 C_zn = pidstd(Kp_zn, Ti_zn, Td_zn, Inf);
39 G_cl_zn = feedback(C_zn * G, 1);
40 step(G_cl_zn);
41 grid on;
42 title('Ziegler-Nichols 整定 PID 控制下的阶跃响应');
43 xlabel('时间');
44 ylabel('幅值');
45 saveas(gcf, 'results/figures/Task02_Figure_2.png');
46
47 %% 3. 微调参数以获得更好的响应
48 figure;
49
50 Kp_tune = Kp_zn * 0.8;
51 Ti_tune = Ti_zn * 0.9;
52 Td_tune = Td_zn * 1.2;
53
54 C_tune = pidstd(Kp_tune, Ti_tune, Td_tune, Inf);
55 G_cl_tune = feedback(C_tune * G, 1);
56 step(G_cl_tune);
57 grid on;
58 title('微调后 PID 控制下的阶跃响应');
59 xlabel('时间');
60 ylabel('幅值');
61 saveas(gcf, 'results/figures/Task02_Figure_3.png');
62
63 %% 4. 性能指标对比
64 info_zn = stepinfo(G_cl_zn);
65 info_tune = stepinfo(G_cl_tune);
66
67 fprintf('\n性能指标对比:\n');
68 fprintf('指标\t\tZ-N PID\t\t微调后 PID\n');
69 fprintf('上升时间(s)\t%.4f\t%.4f\n', info_zn.RiseTime, info_tune.
    RiseTime);
70 fprintf('峰值时间(s)\t%.4f\t%.4f\n', info_zn.PeakTime, info_tune.
    PeakTime);
71 fprintf('超调量(%%)\t%.2f\t%.2f\n', info_zn.Overshoot, info_tune.
    Overshoot);
72 fprintf('调节时间(s)\t%.4f\t%.4f\n', info_zn.SettlingTime, info_tune

```

```

        .SettlingTime);
73
74 %% 5. 分析不同 PID 参数的影响
75 % (1) 不同 Kp 值的影响
76 figure;
77
78 Kp_test = [Kp_zn * 0.5, Kp_zn, Kp_zn * 1.5, Kp_zn * 2];
79 for i = 1:length(Kp_test)
80     C_test = pidstd(Kp_test(i), Ti_zn, Td_zn, Inf);
81     G_cl_test = feedback(C_test * G, 1);
82     step(G_cl_test);
83     hold on;
84 end
85
86 legend('Kp=0.5Kp_zn', 'Kp=Kp_zn', 'Kp=1.5Kp_zn', 'Kp=2Kp_zn', ...
87         'Location', 'southeast');
88 title('不同 Kp 值对阶跃响应的影响');
89 xlabel('时间');
90 ylabel('幅值');
91 grid on;
92 hold off;
93 saveas(gcf, 'results/figures/Task02_Figure_4.png');
94
95 % (2) 不同 Ti 值的影响
96 figure;
97
98 Ti_test = [Ti_zn * 0.5, Ti_zn, Ti_zn * 2, Ti_zn * 4];
99 for i = 1:length(Ti_test)
100     C_test = pidstd(Kp_zn, Ti_test(i), Td_zn, Inf);
101     G_cl_test = feedback(C_test * G, 1);
102     step(G_cl_test);
103     hold on;
104 end
105
106 legend('Ti=0.5Ti_zn', 'Ti=Ti_zn', 'Ti=2Ti_zn', 'Ti=4Ti_zn', ...
107         'Location', 'southeast');
108 title('不同 Ti 值对阶跃响应的影响');
109 xlabel('时间');
110 ylabel('幅值');
111 grid on;
112 hold off;
113 saveas(gcf, 'results/figures/Task02_Figure_5.png');
114
115 % (3) 不同 Td 值的影响
116 figure;
117
118 Td_test = [0, Td_zn * 0.5, Td_zn, Td_zn * 2];
119 for i = 1:length(Td_test)
120     C_test = pidstd(Kp_zn, Ti_zn, Td_test(i), Inf);

```

```

121     G_cl_test = feedback(C_test * G, 1);
122     step(G_cl_test);
123     hold on;
124 end
125
126 legend('Td=0', 'Td=0.5Td_zn', 'Td=Td_zn', 'Td=2Td_zn', ...
127        'Location', 'southeast');
128 title('不同 Td 值对阶跃响应的影响');
129 xlabel('时间');
130 ylabel('幅值');
131 grid on;
132 hold off;
133 saveas(gcf, 'results/figures/Task02_Figure_6.png');
134
135 %% 6. 选择最优参数并显示最终响应
136 figure;
137
138 Kp_opt = Kp_zn * 0.7;
139 Ti_opt = Ti_zn * 0.8;
140 Td_opt = Td_zn * 1.0;
141
142 C_opt = pidstd(Kp_opt, Ti_opt, Td_opt, Inf);
143 G_cl_opt = feedback(C_opt * G, 1);
144 step(G_cl_opt);
145 grid on;
146 title('最优 PID 控制下的阶跃响应');
147 xlabel('时间');
148 ylabel('幅值');
149 saveas(gcf, 'results/figures/Task02_Figure_7.png');
150
151 info_opt = stepinfo(G_cl_opt);
152 fprintf('\n最优 PID 参数及性能:\n');
153 fprintf('Kp = %.4f, Ti = %.4f, Td = %.4f\n', Kp_opt, Ti_opt, Td_opt);
154 fprintf('上升时间 = %.4f\n', info_opt.RiseTime);
155 fprintf('超调量 = %.2f %%\n', info_opt.Overshoot);
156 fprintf('调节时间 = %.4f\n', info_opt.SettlingTime);
157
158 diary off;

```